When the Verifier Cannot Decide: Tracing Math-Verify Failures Through an RLVR Reward Pipeline

Yufeiyang Chen*
School of Cyber Science and Technology, Sun Yat-sen University, Shenzhen, China
Correspondence: fei76161@gmail.com

3 November 2026

Abstract

A mathematical verifier can return zero because an answer is wrong or because verification did not finish. We study whether that distinction survives an RLVR reward pipeline, using a pinned, opt-in Math-Verify path in verl. Persistent-worker injections connect a known sparse-metadata defect to two position-dependent outcomes: a failure flag disappears, or batch assembly raises. All 13 injection cases reproduce on a second platform under real Ray. Porting an open upstream assembly repair preserves the flag in all nine injected rate-limited failure cases, and a further validation-accumulation patch is needed to keep flags aligned across validation batches; both repair observability, not rewards. A fixed-response replay of 93,733 responses records 54 library-event rows and one zero-to-one change under longer limits. The run's corrected report leaves 18 rows unresolved, giving a range of one to nineteen recoveries; the supplied pre-correction records reproduce the recovery and an uncorrected count of 9,573, but not the correction itself, so the range remains conditional on that report. Counting reference-side library errors as unresolved widens it to one to thirty-five. A 22,176-trial replay of human-labelled extracted answers reproduces every historical score on Linux. Natural heavy inputs caused a shared-pool batch abort on the original host under relaxed diagnostic limits; a Linux run with substitute controls did not abort. On 72 complete human-labelled response groups with synthetic failures, zero-filling gives a failed correct answer a negative advantage whenever a correct sibling survives (and zero otherwise); masking only the loss leaves the failed zero in its siblings' baseline; and excluding failed rows from the group statistics avoids that distortion but removes signal from more groups and shrinks the loss. The results argue for testing failure information at the wrapper, assembly and consumer boundaries. They estimate neither training-time incidence nor a learning effect, and longer verification limits are not ground truth.

Keywords: reinforcement learning with verifiable rewards; reward verification; mathematical reasoning; software reliability; large language models.

1 Introduction

RLVR replaces a learned reward model with a program that checks each answer against a reference [5, 15, 24]. For mathematics the check is usually symbolic: the answer and the reference are parsed into SymPy expressions and compared, and a library such as Math-Verify [11] wraps the parsing and comparison in time limits because some expressions are expensive to evaluate. The reward is binary, but the checking program has a third outcome that the binary reward cannot express: it may fail to decide. A comparison can time out, a parser can raise, the subprocess that runs the check can be killed, and a process pool shared by many rows can break.

Training frameworks have repeatedly run into this third outcome. In verl [26], contributors have reported that a signal-based time limit that could not be armed in a Ray worker thread made every MATH reward zero until the limit was disabled [28], that disabling it then let training hang [29], and that a dead or stuck scoring subprocess yields persistent zero rewards [30]. The pull request that addresses the

*The author's primary contributions were selecting the research topic and determining the research direction and methodology. AI agents (Claude and Codex) provided substantial assistance with implementation, experiment execution, code-based analysis and manuscript drafting. The author is responsible for the manuscript and its claims. AI agents (Claude and Codex) also assisted with the evidence audit, the Linux re-executions, the offline policy analysis and the revision of this version.

last report [31] rebuilds a broken pool, kills a stuck one, and propagates persistent failures upward "so the reward manager can flag the row". Separately, contributors have already reported sparse-metadata loss and batch exceptions, proposed union-key assembly in one or both batch paths [7, 14], and reported a downstream metric crash on `None`-filled sparse keys [1, 8]. Other frameworks document the consequence for learning: Open Instruct warns that a fallback zero after a failed grading request "can affect relative advantages as though the response were incorrect" and offers a strict policy that stops the run [2], and NeMo RL lets masked rows be excluded from GRPO group statistics [18]. These records establish known failure mechanisms, candidate repairs and handling policies. They do not by themselves give a rate estimate for the fixed response corpus studied here, or establish how the #8015 wrapper's failure reporting behaves when composed with its manager, aggregation and consumer paths.

We audit the reward pipeline as an end-to-end contract. A wrapper can correctly report an error while aggregation removes that report; a larger timeout can recover a verdict while also keeping costly work alive for longer. Component-level tests therefore need to be connected to row-level batch behaviour. We combine executable mechanism tests, fixed-response replay and matched shared-pool controls to examine these relationships in one pinned configuration. We ask:

RQ1 When the reward wrapper reports a failure, does that information survive the reward manager, batch aggregation and the validation consumer? (§4)

RQ2 Which verifier events and limit-sensitive verdict changes occur in the stored responses, and how do they compare with representation effects in a separate labelled answer set? (§5)

RQ3 How do time limits and concurrent pool recovery interact when naturally heavy inputs enter a scoring batch? (§6)

RQ4 Once a failure is observable, how do common handling policies change group-relative advantages on real response groups? (§8)

The pinned framework comparison (§7) puts these measurements in context; it is a source audit, not a set of additional full-framework experiments, and no section measures learning outcomes.

Contributions. None of the failure mechanisms below is claimed as newly discovered; the added value is the composed trace and the measurements.

1. **A composed failure trace with executable regression requirements.** We trace an already reported sparse-schema failure through the real wrapper, manager and both aggregation functions, using persistent-worker injections. Twenty-one fixed scorer-output fixtures and a larger real-function suite turn the trace into regression requirements. The assembly repair we test is the open upstream proposal #7151, not a new algorithm. In a validation consumer we additionally find that sparse keys are misaligned across validation batches even after assembly is fixed, and give a minimal patch (§4).
2. **A paired measurement of verifier decisions and unresolved cases under a stated rule.** An internally frozen replay covers 93,733 responses and carries unresolved rows into a finite-sample range; we state which reference outcomes the rule treats as unresolved, give the error-inclusive alternative and separate what the supplied raw rows do and do not reproduce. Separate extensions cover all 193,767 stored answers and 5,556 human-labelled extracted answers (§5).
3. **Controlled evidence for timeout–concurrency interactions.** Natural heavy inputs connect the recovery wrapper's previously reported reset interaction to a manager-level batch abort under relaxed diagnostic limits; matched default-limit, serial and single-heavy controls isolate the tested conditions (§6).
4. **An offline comparison of handling policies on real groups.** On 72 complete human-labelled groups and verl's own estimator, we quantify how zero-filling, loss-only masking and exclusion from group statistics change advantages, which rows they retain and how much signal they discard under independent, correctness-correlated and whole-group synthetic failures (§8). The policies themselves already exist in other frameworks; the comparison is a mechanism analysis, not a training result.

Scope. The original experiments ran on one machine (Apple M2 Pro, 10 cores, 16 GiB, macOS, Python 3.12.8) against pinned code; re-executions ran on one Linux x86-64 container (4 vCPU, 15.7 GiB, CPython 3.12.8, CPU only). Framework comparisons are source audits, and the policy analysis is an offline calculation. Rates describe *this filtered dataset re-verified under the declared protocol*, not natural rates during training. The reference judgement is the same verifier with longer time limits, not ground truth. No model was trained.

Table 1: Evidence behind the main results. "Original host" is the macOS machine of the original runs; "Linux" is the re-execution container.

Result	Level	Basis
Position-dependent flag loss or abort (13 cases)	re-executed	original host (2 runs) and Linux, real Ray
21 assembly fixtures	re-executed	original host and Linux
#7151 port, validation patch, chain suite	re-executed	Linux only
Table 5: 93,733 default outcomes, 54 events	recomputed	distributed and supplied rows
$k = 1$; uncorrected $u = 9{,}573$	recomputed	supplied pre-correction rows
Corrected $u = 18$ and its bounds	reported	formal report; event-level reconstruction pending
193,767-answer counts and pairs	recomputed / reported	default rows / report for k, u
Labelled set, Table 6	re-executed	original host and Linux (22,176 trials)
Heavy-input scenes, Table 7	reported; partly re-executed	original host; Linux with substitute controls
Handling-policy comparison, Table 10	offline	72 labelled groups, synthetic failures

Kinds of evidence. Injected failures test whether information survives a particular code path; they do not estimate how often a verifier fails on model output. Stored-response replays supply counts under a fixed scoring protocol, including unresolved cases. Matched heavy-input scenes vary timeout and recovery policies to isolate their effects on a batch, and the offline policy analysis isolates what a handling policy does to advantages. Keeping these roles separate prevents a demonstrated mechanism, a corpus frequency, a diagnostic batch abort and an advantage calculation from being mistaken for one another. Independently of its kind, each number has one of four evidence levels (Table 1): *reported* (read from a report of the original run), *recomputed* (re-derived from distributed or supplied per-row records, not from raw parent/child events), *re-executed* (run again from original inputs on the Linux container), or *offline* (calculated from fixed inputs without running the pipeline).

2 Background

Reward computation in verl. In the pinned verl revision, the default routing sends the MATH datasets to verl's own `math_reward` scorer, not to Math-Verify; the Math-Verify wrapper is an optional scorer that a user selects with a custom reward function, as verl's own tests do. Rewards are computed by a reward manager. The default `naive` manager calls the scoring function in a thread pool (via `run_in_executor`) and does not catch exceptions; the `rate_limited` manager limits concurrency (default 1), and on a timeout or exception records a reward of 0.0 together with an extra `timeout` or `error` key. The default reward configuration starts 8 reward workers. At the tested head no verl code reads the `error` or `timeout` key, and the default GRPO estimator computes group means and standard deviations (`torch.std`, the sample estimate) from all scores of a group before any response mask is applied: the keys are observational metadata only. When rewards are computed during generation, each response is sent to a randomly chosen worker; when they are computed after generation, the batch is split into contiguous chunks. Routing and worker/chunk counts were checked at the tested head `0c4c3d4`; they do not imply eight request threads per worker.

The Math-Verify wrapper in pull request #8015. The wrapper at head `0c4c3d4` keeps a module-level `ProcessPoolExecutor` with at most four spawned workers. It submits each row and waits at most 30 s. On an outer timeout it cancels its own task, kills every worker of the current pool, and returns `timeout_score` (default 0). On the first broken-pool error it rebuilds the pool and retries once; a second consecutive failure is raised to the caller; other exceptions raised in the child propagate unchanged. Python's executor marks every pending future of a broken pool as failed, so one dying worker affects every row in flight on that pool. As of 3 November 2026 #8015 is open with the same head.

Time limits inside Math-Verify. Math-Verify 0.9.0 enforces its own limits with `signal.alarm`: 5 s for parsing each side and 5 s for each candidate pair in the comparison. With `raise_on_error` left at its default (false), a parse that times out or raises returns an empty list and a pair that times out or raises counts as "not equal"; if another pair (for instance the plain-string pair) matches, the overall verdict is still true. The library's timeout exception derives from `BaseException`. Because `signal.alarm` only works on the main thread, calling it from another thread raises an error, which the library rewrites into an explicit message; the wrapper calls the library in the main thread of its pool subprocesses.

3 Verdicts, failures and contracts

We distinguish four kinds of zero (Table 2). This study traces undecided verdicts and reference-side failures through the scoring pipeline.

Table 2: What a zero reward can mean.

Event	Example	Responsibility
Ordinary negative	declared unequal, or no answer admitted by the extraction policy	policy
Policy over budget	truncated response, exhausted turns	policy
Verifier undecided	library timeout or exception, dead worker, broken pool	infrastructure
Data-side failure	the reference answer itself cannot be parsed	data

Whether a zero for an undecided verdict is a *defect* depends on the contract the pipeline claims to implement. Under a *budget contract* ("correct means provably correct within the compute budget") a timeout scored as wrong is intended. An *observability contract* instead requires a failed attempt to remain distinguishable from a negative verdict, even when both receive zero. A separate *exclusion policy* must specify whether an undecided row is removed from the group statistics, the loss, and its denominator. A promise to flag a failure does not by itself promise to exclude it from optimisation. We call behaviour a defect only when a pinned implementation states a contract in code comments, documentation or a pull-request description and its runtime behaviour departs from it. Where no contract is stated we report a *contract ambiguity*. Throughout the replay sections, *default protocol* means the selected Math-Verify wrapper with its unchanged 5 s library and 30 s outer limits on the stated host; it does not mean verl's default scorer routing or a default training deployment. *Diagnostic protocols* deliberately change limits, concurrency or inputs to expose mechanisms. Their outcomes are not reported as default-deployment behaviour. The uniform-key expectation stated in the discussion of #6545 [7] is relevant here: our question is how that assumption composes with a manager that emits failure-only keys. We test the specified returned reward and metadata views, rather than assuming that every sparse dictionary is valid under every consumer's contract.

Operational outcome classes. The replay in §5 uses a narrower vocabulary than Table 2. Table 3 states how each recorded outcome enters its metric; it reproduces the frozen rule (`rewardfail/s2/report.py`) and does not change it. In particular, a reference-side library *error* without a timeout is scored 0 and is not counted as unresolved, so u is not the number of all execution failures.

4 What happens to a failure flag

Pull request #8015 states an observability contract for persistent failures: the wrapper raises "so the reward manager can flag the row", and the `rate_limited` manager does write an `error` key. We tested whether the flag reaches the consumer.

Setup. We ran verl at the pull-request head with the real `rate_limited` manager, the real reward-loop workers and both aggregation functions (the per-sample path used during generation and the batch path used after it). Generation was replaced by fixed response texts, so no model was loaded. Persistent failure was injected by making the scoring subprocess call `os._exit(17)` on a marked response, on every attempt; a ledger recorded each injection. Thirteen cases varied the position of the failing row, whether whole

Table 3: How a recorded outcome enters the replay metric. "Event" means Math-Verify logged a timeout or an error for a step; because a failed parse returns an empty list and a raised comparison counts as unequal, an event can coexist with score 1.

Recorded class	Default pass	Reference pass (default-0 row)
score 1 (with or without event)	1	recovery, counted in k
score 0, no event	0	stays 0
score 0, library *timeout* (parse or compare)	0	unresolved, counted in u
score 0, library *error* (parse or compare raised)	0	stays 0, not in u
outer timeout, broken pool, wrapper exception	0	unresolved (none observed)
interrupted twice, resource-guard stop, incomplete records	unresolved	unresolved, counted in u

chunks or whole batches failed, and the manager. The two recorded runs are in `results/c1c/run1/` and `results/c1c/run2/`, against execution commit `cc87848`. They expose the same outcomes on repeated execution of the declared cases. The unchanged driver was also re-run on Linux under a real local Ray session; all 13 cases reproduced the historical outcome, exception type, raise site, key set and per-row scores.

Results. Both aggregation paths take the set of extra-information keys from the *first* row of a chunk and then index every row by those keys. This is the sparse-schema mechanism already described in [7, 14]. Our failure injection makes the incompatible manager outputs and their returned failure information explicit:

- If the failing row is not first, the batch completes, the failing row contributes 0, and the reward returned by `extract_reward` carries no failure flag: from that return value a failed scoring is indistinguishable from a wrong answer. (The raw per-sample column on the generation path still holds the flag; we did not run a full trainer to see whether anything reads it.)
- If the failing row is first and other rows are normal, aggregation raises `KeyError` and the batch aborts. Moving the *same* originally correct answer from a later position to the first reproduced the switch, so position, not content, decides between the two outcomes.
- If an entire chunk fails, mismatched keys or metadata across chunks trigger assertion failures (`py_functional.py:366`, `protocol.py:949`); if the entire batch fails, zeros and flags are returned consistently.
- Under the default `naive` manager the same failure aborts the batch with the true cause, because the manager does not catch it. The flagging design therefore only applies with `rate_limited`.

Wrapper exception ⟶ manager returns reward 0 and an error flag

Failed row	Row 1	Row 2	Row 3	First-row-key aggregation
Row 2: rewards	1	0	1	first row has no error key
Error metadata	—	error	—	error column is omitted
Row 1: rewards	0	1	1	first row contains the error key
Error metadata	error	—	—	indexing a normal row raises `KeyError`

Union-and-fill aggregation: retain the key and align missing entries explicitly

Figure 1: Schematic of the row-position effect. Common metadata is omitted; three rows illustrate the first-row key rule rather than a new measured batch. Both real aggregation paths exhibit this effect. The tested repair prototype applies only to batch assembly after scoring, with missing entries represented by `None`.

Row-preserving aggregation. Union-key assembly with explicit missing entries was proposed upstream [7, 14]; it is not a new repair algorithm. We use a bounded local version to specify and test the failure-recording contract. Let a batch contain ordered pairs (r_i, e_i), where r_i is the numerical reward and e_i is the row's optional metadata dictionary. An aggregation that preserves failure information must satisfy three requirements:

1. *Key completeness:* retain every key in $K = \bigcup_i \text{keys}(e_i)$, including keys absent from the first row.
2. *Row alignment:* each metadata column has one entry per returned reward, in the same order after any padding, trimming, chunking and concatenation. Absence requires an explicit representation rather than dropping the column.
3. *Reward preservation:* adding observational metadata leaves the numerical reward vector unchanged; exclusion from an update is a separate policy decision.

The first-row key rule violates completeness, and direct indexing assumes every row has every retained key. The fixtures below exercise these two failure modes at each row position and check alignment across worker chunks. These are aggregation requirements; they cannot expose a library failure that was never included in e_i.

Row-permutation equivariance. For a batch without padding, the same requirements imply an executable metamorphic check: reordering fixed scorer outputs should only reorder the returned rewards and metadata. If π is a row permutation and $A(B)$ assembles batch B, then for every retained metadata key k,

$$\text{scores}(A(\pi B)) = \pi\,\text{scores}(A(B)), \qquad \text{column}_k(A(\pi B)) = \pi\,\text{column}_k(A(B)).$$

The retained key set should be unchanged. Moving a flagged row to the first position violates this requirement in the original code even though its scorer output has not changed. The eight error-position and eight timeout-position fixtures below exercise the same row-alignment requirement for the local prototype.

A bounded aggregation prototype. We tested a local prototype on the pinned batch aggregation method, changing only two assignments: take the (sorted) union of extra keys over all rows, and fill missing entries with `None` in object-valued columns. Twenty-one fixed scorer-output fixtures cover an `error` or `timeout` at each of eight positions, mixed flags, empty extras, normal outputs and padding/trimming across two workers. The original method omits columns in 16 fixtures and raises in three; the prototype returns the expected reward vector and aligned extra values in all 21. Upstream source was unchanged. Re-executing the companion's own fixture code on Linux gave the same source hash, prototype AST hash and all 42 outcomes. The prototype covers only the batch path, converts every column, including fully populated numeric ones, to `dtype=object` and sorts the keys; it is a regression target, not a candidate upstream change. It does not repair naive-manager exceptions or silent library failures.

Upstream repair and a remaining consumer gap. The open pull request #7151 [14] implements a shared assembler for both paths: first-seen key order, `None` only in sparse columns, natural dtype for fully populated columns, and agent-loop assembly only after all worker chunks are concatenated. Its diff applies cleanly to `0c4c3d4`; its own CPU tests pass on the ported tree (21 passed) and fail on the unpatched tree (7 failed, 6 passed, one file not importable). Extending the trace to the consumer, `RayPPOTrainer._validate` at the same head still accumulates extra keys per validation batch without padding: a key that appears only in a later (or only in an earlier) validation batch leaves lists shorter than the sample list, and the length assertion at `ray_trainer.py:708` aborts validation; `_merge_validation_results` concatenates such lists without padding and shifts a flag to the wrong sample. A small patch back-fills and pads with `None`. `process_validation_metrics` at this head drops `None` before averaging, so the crash reported in #6830 [8] did not reproduce on that path; we did not test the other trainer entry point named in the report. Upstream `main` checked on 3 November 2026 (`2bcc98a`) still uses the first-row key rule and the unpadded validation accumulation. A limited keyword search found no upstream report of the validation misalignment; we do not claim it is a new failure class.

Re-running the 13 injection cases under real Ray against the patched tree, all 11 `rate_limited` cases completed and the `error` column was present in all nine with an injected failure; per-row rewards equal those of the unpatched tree wherever the latter completed. The two `naive`-manager cases still abort with `BrokenProcessPool`, which these patches do not address. Missing and explicit `None` values remain indistinguishable after assembly, and no consumer at this head acts on the flag: a flagged row still enters

Table 4: Real-function regression suite (Linux). Each row runs the real verl functions of the named tree; substitutes are fixed response texts, a text tokenizer, in-process stand-ins for Ray dispatch and stub trainer attributes around the real `_validate`. This is a minimal integration of real functions, not a full trainer run.

Contract (tests)	`0c4c3d4`	+ #7151 port + validation patch
Wrapper: correct/wrong, transient and persistent worker exit, outer timeout, ordinary and library-timeout exceptions (6)	pass	pass
Managers: `rate_limited` error and timeout flags; `naive` batch abort (3)	pass	pass
Batch path: flag at each of 8 positions, several sparse keys, none/all flagged, single row, 40 permutations, four chunk/padding layouts, schema split across workers, dtype kept (11)	3 tests and 8 sub-cases fail	pass
Generation path: each position; flag only in first or second worker (3)	all fail	pass
Consumers: `_validate` with sparse keys within and across batches, merge alignment, `None` in metrics (6)	4 fail	pass

GRPO statistics and the loss as a zero, which is the default "include" behaviour that NeMo RL makes configurable [18].

Library-internal timeouts do not reach the wrapper's timeout branch. The wrapper maps the library's timeout exception to `timeout_score`. With default parameters the library catches its own timeouts first, so the branch is not reached: setting `timeout_score` to -7 left library-level failures at 0 or 1, and only the outer 30-second wait and an explicitly raised timeout returned -7. In a controlled slow comparison, two originally correct answers dropped to 0, while a third still scored 1 through the string fallback. Because no contract states which timeout layers `timeout_score` is meant to cover, we report this as a contract ambiguity, not a defect.

Mechanism evidence and replay incidence. The aggregation behaviour is a property of real verl code at the tested head, exercised end to end through the manager. The failure itself was injected; it does not show that natural inputs reach the propagation path. The next section measures what natural inputs do.

5 Fixed-corpus measurement: re-verifying 93,733 responses

5.1 Protocol

The sample, replay settings, observation checks, follow-up selection and decision rule were fixed in the author's experiment repository before the main sample was scored (execution snapshot `9a88991`). That repository is not public, so the freeze is internal and not independently timestamped; a code snapshot of `9a88991` has since been supplied with the author's inputs. The sample map and threshold calculations are distributed under `plan/` in the companion archive. The finite-sample recovery range below is an additional interpretation of the same frozen counts, not a change to the pre-declared rule.

Sample. OpenR1-Math-220k [12] (configuration `default`) contains 93,733 problems from Numina-Math 1.5, each with one to six reasoning traces generated by DeepSeek-R1; according to the dataset card, traces were verified with Math-Verify or, for about 12% of samples, an LLM judge, and every problem has at least one trace judged correct. For each problem we chose one response uniformly with a fixed seed, using a hash of the problem text (the dataset's `uuid` is missing for 664 rows and shared by 60 pairs of different problems). Question identity is exact text; near-duplicate problems with different text count separately. The full response text, including the reasoning, was passed to the wrapper with the dataset's `answer` as reference. The dataset was already filtered by the same library, so nothing about the direction of bias can be assumed.

Isolated replay. Each measurement process owns one wrapper and its pool, submits one row, waits for it, and only then submits the next; a new pool is first warmed with a trivial row whose result is discarded. There is therefore no queueing and no cross-row interference by design. The wrapper code, the library limits (5 s), the outer limit (30 s) and `timeout_score`= 0 are unchanged.

Observation layer. In the parent we wrap pool creation, task submission and each wait, recording outcomes, pool generations, resets and every kill. In the child, before the unchanged worker function runs, we wrap Math-Verify's `parse` and `verify` and record each step (reference parse, answer parse, each comparison pair) with wall and CPU time, the result type, library log records, and the arrival of every alarm signal. Records carry sequence numbers; a synthesis step cross-checks parent and child records row by row and marks any row whose records are incomplete or inconsistent as unresolved. Turning the layer off and on changed no score on 500 pilot rows. Eighteen negative controls, each corrupting one record type, were all detected. The self-test records are in `results/s2_rev2/selftest/report.json`.

Reference judgement and follow-ups. Every row scored 0 by default was re-scored with the library limits raised to 60 s and the outer limit to 300 s (the *reference*). A pre-drawn random set of 1,000 default-1 rows was also re-scored to check for one-to-zero changes in those sampled rows. Rows with a default or reference event, a score change, or a limit-sensitive change were replayed three times under the default setting. Rows with a library timeout or any step over 1 s, plus a pre-drawn random 1%, were re-scored with library limits of 2 s and 10 s. Another pre-drawn 1% of rows were re-run unchanged to measure test–retest instability.

Metric and decision rule. Let k be the number of rows scored 0 by default and 1 by the reference, and u the number left unresolved under the rule of Table 3 (the reference itself timed out, the row was interrupted twice, or the local resource guard stopped it). A reference-side library error without a timeout is scored 0 and is not in u. The denominator is always $n = 93{,}733$. For the realised sample, the zero-to-one recovery share over completions of the recorded reference outcomes has the following worst-case range:

$$\frac{k}{n} \;\le\; q_{\text{recover}} \;\le\; \frac{k+u}{n}.$$

Here a completion holds every resolved recorded score fixed and assigns each of the u unresolved default-zero rows either score 0 or score 1. The endpoints follow by assigning all such rows 0 or all 1; every intermediate integer count is possible absent further information. This is a missing-outcome sensitivity calculation, not a prediction that more time will resolve these rows, nor a bound on the outcome of a new run in which already-resolved scores may also change. It needs no assumption of independent, identically distributed problems and describes only this realised sample. Numerical endpoints using the corrected u remain conditional on the historical report until its event-level reconstruction is available. They concern score recovery under the specified verifier, not mathematical correctness. The pre-declared decision rule additionally uses binomial-model Clopper–Pearson bounds [4]. These are working-model bounds for the declared rule: they treat the 93,733 rows as independent draws with a common rate, which the corpus does not guarantee, and they are not distribution-free bounds for heterogeneous problems or guarantees about a training population. The two kinds of interval answer different questions and are reported separately. Before the run we fixed: continue to the consequence stage if the one-sided 95% lower bound on k/n is at least 10^{-3} (here $k \ge 111$), or if a natural input reproducibly raises through the wrapper, or if an outer timeout spills over to other rows; stop this line if none holds and the one-sided 95% upper bound on $(k+u)/n$ is below 10^{-4} (here $k+u \le 4$); otherwise report the result as *in between*.

5.2 Results

Events are rare; selected replays are stable. Under the default protocol 45 zero-scored rows had a verifier-side event (Table 5); a further 9 rows had a comparison error in one candidate pair but still scored 1 through another pair. No row hit the outer 30-second limit, broke the pool, or made the wrapper raise, and no row took more than 10 s. All 47 rows selected for replay gave the same outcome three times out of three, and the 963-row retest changed nothing.

One score change, eighteen unresolved (formal report). The reference turned exactly one default 0 into a 1 ($k = 1$). Eighteen rows stayed unresolved: fourteen in which the 60-second comparison

Table 5: Default-protocol outcomes on the main sample ($n = 93{,}733$). Recomputed from the distributed derived rows and from the supplied raw rows, re-deriving each class from the stored step records: all counts agree, ids are unique and no row has a second attempt.

Outcome	Rows
Score 1	61,246
Score 0, no verifier event	32,442
Score 0, comparison timed out (library)	27
Score 0, comparison raised (library)	12
Score 0, answer parse raised (library)	4
Score 0, answer parse timed out (library)	1
Score 0, reference parse timed out (library)	1
Outer timeout, broken pool, exception raised by the wrapper	0

also timed out, one in which parsing the reference answer timed out, two interrupted twice and one stopped by the local resource guard. The last three were interrupted or stopped by the resource guard then in use on this 16 GiB machine (since retired); they are local interventions, not natural outcomes. All 1,000 default-1 rows stayed 1. The finite-sample recovery interval is $[1.07 \times 10^{-5},\ 2.03 \times 10^{-4}]$. With $k = 1$, $u = 18$ the one-sided 95% binomial-model upper bound on $(k + u)/n$ is 2.97×10^{-4}; the two-sided 95% interval for k/n is $[2.7 \times 10^{-7}, 5.9 \times 10^{-5}]$. Neither threshold is met, so the pre-declared verdict is *in between*. Of 928 rows re-scored with 2 s and 10 s limits, three changed with the limit: two scored 0 at 2 s and 1 at 5 s and 10 s, and one (the k row) scored 1 only at 10 s.

What the supplied records reproduce. The formal report is a re-analysis. The 7 October analysis attached every local resource-guard termination to all rows that the killed pool process had scored. The supplied rows contain 9,868 guard-stopped reference rows (9,864 excluding warm-ups); the report records 9,866 reclassifications (9,863 excluding warm-ups), leaving one non-warm-up row guard-stopped. The corrected `analyze.py` (SHA-256 `f6e70c6c...`) attaches a termination only to attempts whose result had not returned at least 1 s before the kill. The supplied pre-correction rows contain 93,733 default and 33,487 reference rows (32,487 default zeros plus 1,000 controls); paired under the frozen rule they give $k = 1$, with the same row as the formal report, and $u = 9{,}573$, of which 9,559 are guard-stopped rows. The reference pass recorded only four guard termination events. Applying the reported all-row class changes to the pre-correction all-row totals reproduces the diagnostic report totals. Those totals exceed the formal non-warm-up totals by six score-one rows and one guard-stopped row, consistent with the seven warm-ups. This is aggregate, report-level reconciliation, not reconstruction of the corrected warm-up or measurement rows: the corrected per-row classes, and hence $u = 18$, depend on parent, child and guard event timing that is not in the supplied compact rows, and we did not infer them from the formal report.

Error-inclusive sensitivity. After correction the reference pass also has 16 zeros with a library error but no timeout (12 comparison errors, 4 answer-parse errors), which the frozen rule scores as 0. Because all 1,000 controls scored 1 under the reference, these are default zeros disjoint from k and u. Counting them as unresolved gives $u_{\text{err}} = 34$, a finite-sample range of $[1/n,\ 35/n] = [1.07 \times 10^{-5},\ 3.73 \times 10^{-4}]$ and a binomial-model upper bound of 4.95×10^{-4}; the pre-declared verdict is unchanged. This uses the formal aggregate classes; row pairing after correction could not be verified.

The bound rests on the reference not being backfilled. Had all eighteen unresolved rows scored 0 under some longer limit, the upper bound would fall below 10^{-4}; had all scored 1, k would be 19, still far from the continue threshold. The supplementary diagnostics below were specified in advance for exactly these rows and are reported separately; they do not change the formal result.

5.3 The one change, measured again

The single k row changes from 0 to 1 when the comparison limit is relaxed; this score change is not an independent proof of mathematical correctness. Re-run ten times in sequence on one warmed pool, it timed out at the 5 s alarm every time (comparison wall 5.002–5.005 s) and scored 0; under the reference it scored 1 every time with comparison wall 5.82–7.51 s (median 6.21 s). The ten repeats share a pool and SymPy's caches and are not independent samples, and we have no measurement on other hardware;

all ten repeats show the same limit-dependent pattern on this machine. The first repeat had a gold-parse time of 0.11 s against <0.001 s afterwards; cache state was not controlled. It behaved the same way: 0 at the 5.005 s alarm under the default, and 1 under the reference with the slowest comparison of the ten (7.51 s). Whether this row changes on faster or slower hardware is untested; its margin over the alarm is hardware-dependent.

5.4 Heavy natural inputs

Four unresolved rows were singled out because the reference either could not finish them or stopped them for memory. We ran each in a fresh process that calls the wrapper's worker function directly with the 60-second library limits, dumped all thread stacks every 10 s, and stopped a run at 1.5 GiB of memory footprint or 120 s of wall time after a final stack dump. These are stack-capture endpoints, not resource limits of the method.

All four stacks end in the same place: Math-Verify's numeric equality calls SymPy's `doit()`, which evaluates an integer power in `Integer._eval_power`. Three rows express the same recursive-function task for $f(4, 1981)$, with identical recurrence rules. Their *reference answers* contain nested powers: $-3 + 2^{2^{1984}}$, $2^{2^{1984}} - 3$ and $2^{2^{1984}-3}$. The last is not the same expression as the first two; we report the corpus strings rather than settling their mathematical correctness. The responses all answer $2 \uparrow\uparrow 1984 - 3$. In each, footprint grew at 82–88 MiB/s over the first 15 s and passed the 1.5 GiB endpoint within about 18 s of process start, with no sign of levelling off within that window; a truncated trace says nothing about whether or when the computation would end. The fourth row's reference answer is 0 and its response contains $5^{5^{5^5}}$; there memory grew slowly (peak 351 MiB) and the 60-second alarm stopped the comparison on time, scoring 0, as in the main measurement. Under the default 5-second limit all four were stopped by the alarm and scored 0; none reached the outer limit or harmed the pool.

So in the default protocol the library's alarm is what contains these inputs: they score 0 in about 5 s and leave the pool intact. Whether the 0 is wrong is not settled here; all four rows remain unresolved. The trigger can sit in the reference answer, in which case any response to that problem whose comparison takes the same numeric path would meet it — in GRPO, possibly several members of one group. The corpus extension below counts within-question events under isolated replay; it does not reproduce simultaneous generation or training. The shared-pool experiment below tests selected inputs at explicitly relaxed limits; disabled limits are only a source-level observation.

5.5 The remaining fourteen

The other fourteen unresolved rows were re-scored once each with 300 s library limits (outer limit 1,500 s), a bounded sensitivity analysis rather than a ground truth. None scored 1. Eleven comparisons reached their 300 s wall-clock limit and one reference-answer parse reached that limit; the limit applies to each library operation, not to the entire response. Two comparisons finished and returned false, after 81 s and 281 s. Counting the four heavy rows as still unresolved, this gives $k_{300} = 1$ and $u_{300} = 16$: the one-sided 95% upper bound on $(k_{300} + u_{300})/n$ is 2.72×10^{-4} and the lower bound on k_{300}/n is 5.5×10^{-7}, so the analysis stays between the two pre-declared thresholds, as the formal result does. A preliminary reading of the answer strings, which is not ground truth, judged ten of the twelve rows that remain unresolved to state the reference answer and the other two not to; it agrees with both finished zeros (one reading "different", one with nothing to judge). A limit sixty times the default thus decided two of the fourteen, both as zeros.

The run was interrupted once after seven rows and resumed from the same directory; the eighth row was re-run from the start. During the first part the machine carried heavy unrelated load (median one-minute load average 10.5), and one memory-pressure reading was critical, which paused dispatch for 32 s without stopping the row in progress (the one that finished at 281 s, with 208 s of comparison CPU time). The eleven timed-out comparisons used approximately 250–299 s of CPU time. These results show that the stated wall-clock protocol did not finish those operations on this host; they do not establish a hardware-independent minimum computation time.

5.6 Extension to all stored long-form answers

After the main result was fixed, we specified a replay of only the 100,034 answers outside the one-answer-per-question sample. Question identity is the exact text's SHA-256, with an answer index appended; the 93,733 question strings contain one to six answers each, 193,767 in total. The original sample, its

formal report and decision thresholds were not re-scored or changed. Each new answer used the same isolated 5 s / 30 s protocol; all 32,704 answers not scoring one were re-run at 60 s / 300 s, along with 1,000 default-one answers selected by a fixed hash ordering of question key and answer index. Code, data, selection and baseline hashes were locked before replay.

The complement contains one default-zero/reference-one change and fourteen unresolved answers (thirteen comparison timeouts and one reference-parse timeout). All 1,000 controls remain one. Joining these *counts* to the historical main report gives two changes and 32 unresolved answers in 193,767: the corresponding finite-corpus proportions are $2/n$ and $34/n$, or 0.00103% and 0.01755%. These are descriptive endpoints, not confidence bounds, and do not apply the main sample's decision rule. They inherit the unverified event-level correction underlying the main report's $u = 18$. The second change is another answer to the main sample's one limit-sensitive question; it is not a second distinct problem exhibiting that effect. Likewise the 32 unresolved answers come from only 21 distinct questions (11 questions contribute one unresolved answer to each of the two samples); counted per question, the pooled endpoints are 1 and 21 of 93,733 questions (recounted from the reported unresolved identities, not reconstructed from corrected raw events). Answer-level and question-level proportions answer different questions and should not be mixed.

The full default replay has 103 answers with an observed library failure event; 85 score zero and eighteen still score one through another accepted comparison. Of the 92,287 questions with at least two answers, 59 have an observed event and 42 have events on two answers. Define an ordered pair by choosing an observed-event answer and one other stored answer to the same exact question. There are 103 such pairs, of which 84 have events at both endpoints (81.55%); no other endpoint has incomplete observation. This denominator excludes single-answer questions and weights questions by the number of event anchors and other answers. All 42 multi-event questions have two stored answers. The same reference and question content, different replay times and uncontrolled caches can jointly explain co-occurrence. We do not estimate a correlation parameter, a training failure rate or dependence in eight-answer GRPO groups from these counts.

The default complement replay finished in 14.2 minutes. Its reference controller disappeared once with one unfinished answer; the reason was not recorded. The same directory and version lock were resumed, retaining completed work and the interrupted attempt. The reported 20.4-minute reference duration describes the resumed controller only, not all reference work or the intervening gap. No red-pressure intervention occurred in this extension (maximum recorded pressure was warning). On the original host, raw-event reassembly and a separate count/pair reconstruction matched the stored report. Recomputed from the distributed default rows, the question sizes, the 92,287 eligible questions, the 59/42 questions and the 103/84 pairs all agree; main and complement ids are disjoint and every complement answer maps to a main question.

5.7 Cross-check with human-labelled extracted answers

We used the released human-labelled 1.5B-model answers of Xin [33, 34]: 5,556 answers, with 2,822 positive and 2,734 negative labels, drawn from 3,041 groups. The author's sampling code stratifies by category and verifier opinions; this is not a random sample of a training population. Only 72 groups contain all eight answer positions. Two annotation columns agree on the 200 double-labelled rows; we did not independently adjudicate the labels. The input is an extracted answer, not the full generated reasoning used in the main sample.

Each answer was replayed in isolation through the same pinned wrapper, first verbatim and then, if it did not already contain `\boxed`, enclosed in `\boxed{...}`. Each presentation was scored with default 5 s library / 30 s outer limits and with 60 s / 300 s limits. The 24 already-boxed answers reuse their recorded scores rather than creating a second identical trial. Thus there are 22,176 actual trials and 5,556 label-aligned rows per presentation and limit condition. Input strings, source hashes and alias mappings were fixed before measurement. All rows have complete observation records, and no resource intervention occurred. An ordinary zero here means no observed failure event, not necessarily a completed unequal comparison: an empty extraction under the specified format policy also produces zero.

Under either presentation, no observed failure-event zero intersects a positive human label (0/2,822 label-positive answers). Extending the limit eliminates one comparison-timeout event per presentation but changes no score. This is a zero result on the selected sample; it is not a proof that timeouts cannot hide correct answers elsewhere. The presentation change turns 1,431 zeros into ones and nine ones into zeros, reducing disagreement but increasing label-negative ones from 43 to 91. Most disagreement therefore cannot be attributed to the observed timeout events. The delimiter condition is an explicit

Presentation	Label-positive / score 0	Label-negative / score 1	Disagreement	Event zeros
Verbatim	1,487	43	1,530	3 / 2
Explicit delimiter	113	91	204	4 / 3

Table 6: Label disagreement at default and extended limits: the scores are identical between the two limits. The last column gives default / extended failure-event counts; all those answers have negative human labels.

input transformation, not a repair demonstrated on full generated responses or a new verifier method. The underlying label study already analyses verifier disagreement and group dependence; our addition here is the failure trace under the specified execution protocols.

New Linux replay from the public input. Because the labelled CSV is public, we re-ran this cross-check from its original input under a protocol fixed before scoring: the CSV re-downloaded from the pinned commit (SHA-256 matched), the unchanged #8015 wrapper, and a simpler child-side observer that executes the same `parse` and `verify` calls and captures Math-Verify's log records. All 22,176 trials ran sequentially in 11.9 minutes with no exception, outer timeout or incomplete record. Every trial's score equals the historical score for the same row and condition, and the rows with a logged timeout or error are the same rows in all four passes (3, 4, 2 and 3 rows), so Table 6 is reproduced exactly on a second platform. This concerns this extracted-answer set only; it says nothing about the long-form corpus, timing margins such as the single k row, or label validity.

6 Consequences of heavy natural inputs

6.1 Matched single-heavy batches

The two unchanged wrappers were compared at `0c4c3d4` and `acad987`, with a process-pool initializer setting the library limits inside each spawned child's main thread. The original scoring functions, manager and aggregation functions were executed; fixed text, an index text-tokenizer and a local substitute for Ray transport replaced generation and remote dispatch. One naive manager used eight request threads and 64 rows. This is a controlled amplification at one reward actor, not the full default split across eight actors. Four extra score-one answers warmed each fresh scene sequentially; this does not guarantee that all four lazy pool children had been warmed. The manager is instantiated afresh for each batch, while the scene retains its event loop, thread executor and wrapper pool. Following batches therefore probe scorer/pool state, not survival of the same manager object.

The matrix comprises two wrappers, two library limits (unmodified 5 s or 60 s), and fixed 30 s outer waits. The 64 controls were selected by a fixed hash ordering from the previous score-one set, independently of these new outcomes. For each wrapper/limit pair, an all-control batch ran once; the response-heavy input at position 1 or 33 and one reference-heavy input at position 1 each ran three times. Every scene ended with a following 64-control batch in the same pool. The 40 scenes had no local resource intervention; maximum recorded pressure was normal. Synthetic tests first checked on/off score equality, default versus relaxed library timeouts and the two outer-timeout pool policies. The natural scenes used the specified 30 s waits, not the synthetic test's shortened wait.

Every heavy row scored zero, all 63 unaffected controls scored one, and all following controls scored one. Under the 5 s library limit, the heavy scoring calls returned after 5.03–5.34 s with no pool reset. Under the 60 s library / 30 s outer diagnostic, the heavy call returned zero at 30.00–30.18 s in both wrappers. The pull-request wrapper reset the pool in all nine heavy scenes at this diagnostic setting; the main wrapper reset it in none. In main, the child's heavy computation ended approximately 29.72–29.92 s *after* the following control batch had finished, showing that returning zero did not stop the background work. Its scene shutdown waited for that work; it was not an observed training stall.

No shared-pool score damage was observed with *one* heavy row in these particular batches. This is not evidence that arbitrary concurrent inputs are safe. More controls could consume the remaining pool capacity, and the default concurrent protocol could reach the outer deadline by queueing or cumulative comparison work. The diagnostic distinguishes cleanup policies, not the deployment frequency of a default fault.

6.2 Three-heavy, eight-row manager controls

A separately specified matrix used the first eight of the same control answers. For each wrapper and library limit, both a naive manager with eight request threads and a rate-limited manager at its default concurrency one scored either a pure-control batch or a batch with the three reference-heavy rows first. Every scene then scored eight controls. All 16 scenes completed without resource intervention. This remains one locally invoked reward actor; it does not execute a full Ray trainer.

Library limit	Manager	Recovery wrapper	Main wrapper
5 s	Naive	Three zeros, five ones	Three zeros, five ones
5 s	Rate-limited (1)	Three zeros, five ones	Three zeros, five ones
60 s	Naive	BrokenProcessPool; no batch return	Three zeros, five ones
60 s	Rate-limited (1)	Three zeros, five ones	Three zeros, five ones

Table 7: Three natural reference-heavy inputs in an eight-row batch, with 30 s outer waits throughout (original host). Pure controls and following controls all score one. The 60 s condition is a diagnostic, not a library default.

The pull request's public discussion had already reported a second `BrokenProcessPool` under concurrent resets with a minimal verifier stand-in, and its author explicitly retained that propagation [25]. Our experiment extends that engineering record with real verifier inputs and the manager's batch outcome; it does not claim to discover the reset interaction. The diagnostic naive recovery-wrapper scene raised `BrokenProcessPool` after repeated pool resets; its five control calls individually returned one, but no aggregate batch reward was returned. The traceback reaches the wrapper's persistent broken-pool raise. Two additional traced repetitions reproduced the batch abort (three traced runs in total). A fourth run disabled child step/DEBUG observation and also aborted; parent scheduling logs and the time-limit initializer remained in place, so this is not a completely uninstrumented run. The four matched main-wrapper scenes returned three zeros and five ones. No subprocess-exit fault was injected, and no red-pressure guard intervened. Default-limit and serial controls in the initial matrix each ran once; we do not assign them a frequency estimate.

On a batch exception, the harness records it, drains requests already started by the real gather, and only then issues the controlled following batch. Those batches all return eight ones. This demonstrates recovery under that harness, not that an actual training loop automatically resumes. The contrast is a consequence of timeout/recovery policy interacting with concurrency: it is not evidence that the declared zero-on-timeout policy is itself a defect, nor that the diagnostic abort occurs at default limits. It also does not establish deployment incidence or satisfy the original main-sample decision rule retrospectively. The formal result and thresholds remain unchanged.

Linux re-run of the heavy-input scenes. We re-ran eleven scenes on Linux with the companion's unchanged scene runner and the three reference-heavy texts, but with eight short synthetic controls in place of the dataset controls, which were unavailable there. Default-limit, rate-limited and pure-control scenes agreed with Table 7: three zeros, five ones and following controls all one. The diagnostic naive scene with the recovery wrapper at 60 s / 30 s did *not* abort in any of four runs (three pool resets each, batch returned), including one on the patched tree. Its timeline shows that every control had returned about 20 s before the three heavy waits expired within 10 ms of each other, so each reset met only futures that had already timed out. The observed macOS traceback is consistent with a reset breaking a still-pending future. That opportunity depends on scheduling and control duration. Because the Linux run changes both platform and control inputs, it does not isolate either cause of the discrepancy. It fails to reproduce the historical abort and shows that these heavy inputs do not guarantee an abort under the tested diagnostic limits.

6.3 Sequential parent calls and a longer outer wait

Fourteen additional, separately specified scenes reused the first eight controls, the three reference-heavy inputs and the four warm-up answers. Each wrapper had seven one-time scenes: pure-control and three-heavy inputs under direct sequential calls at 5 s / 30 s and 60 s / 30 s; and pure-control, single-heavy and three-heavy inputs under the eight-thread naive manager at 60 s / 300 s. Each scene ended with

eight controls. All fourteen completed, with no resource intervention and maximum pressure normal. Every normal answer scored one, and every heavy answer scored zero.

Under sequential parent calls at 60 s / 30 s, the recovery wrapper reset its pool three times but returned all row results. Main made no reset; its child's work continued after each outer return. The recorded child intervals overlap even though the parent invokes the wrapper sequentially, so this control does not imply entirely serial background computation. At 60 s / 300 s the library's comparison deadlines ended the heavy work before the outer waits; neither wrapper reset its pool, both naive batches returned, and all following controls scored one. These controls are consistent with the earlier batch abort arising from concurrent outer-timeout recovery, rather than an isolated heavy call. They do not establish that every concurrency pattern or timing environment behaves the same way.

7 Other frameworks

Scoring paths were inspected at fixed revisions (read 8–9 October 2026). The function-body and string checks below test small parts of those paths; full deployment configurations were not executed.

TRL. TRL's built-in `accuracy_reward` [13] calls Math-Verify in the calling thread, with no worker process. On the main thread it passes a 10 s parsing and a 5 s comparison limit; off the main thread it passes `None` for both, which disables the alarm, and lowers Math-Verify's log level to hide the resulting warnings. An unparseable reference answer yields `None`, which the GRPO trainer turns into NaN and so keeps as a flag; a comparison that times out returns false and becomes an unflagged 0. The GRPO trainer calls synchronous reward functions directly in its training loop, without exception handling. Under an ordinary script launch that loop is the main thread, so the limits apply.

Ray. The Ray Train example for TRL GRPO [23] copies `accuracy_reward` and sets both limits to 0, stating that the default limits are incompatible with Ray. In Math-Verify 0.9.0 a limit of 0 or `None` means no alarm at all. A run that follows this example therefore has no library deadline if a comparison reaches the integer powers of §5.4; the verifier is called inside the training process. This is a statement about the code; we have not run such a configuration.

verl main. At the main-branch revision we read (`acad987`), verl's default scorer [32] routes by data source and does not use Math-Verify: the MATH sources go to a rule-based checker, with Math-Verify offered only in a comment as a manual override; DAPO and AIME sources go to another rule-based checker; six NuminaMath sources go to `prime_math`. Its first symbolic path has an intended 10 s subprocess timeout and catches exceptions as false; we have not executed that path across multiprocessing start methods. The fallback calls `math_equal(..., timeout=True)`, passing a boolean to a duration parameter. More directly, its `symbolic_equal` uses `with timeout_limit(...)` although that helper returns a decorator function. Each such block raises `TypeError` before parsing or comparison, and the exceptions are swallowed. In a bounded check using the fixed function bodies and call sentinels, the fallback returned false without reaching any symbolic operation. Earlier string or numeric branches can still return true; this is not a claim that all prime_math scores are zero. The #8015 wrapper we test therefore sits on the opt-in Math-Verify route, and our results apply to configurations that choose it.

OpenRLHF. OpenRLHF [19] does not use Math-Verify. Its math grader (adapted, per its header, from the OpenRLHF-Agent project) compares with SymPy `simplify` but first refuses to hand SymPy any expression containing `^{` or `^(`, a chained power, an exponent of two or more digits, or more than two unknown letters; such pairs are graded unequal without evaluation. On the main thread `simplify` runs under a 120 s alarm; off the main thread (the code comment names Ray actors) it runs without a limit. A timeout or other exception is graded false, again without a flag. This is a contrasting input admission policy; prime_math's first path contains the same power filters. In a bounded check of the pinned extraction and normalisation helpers, the candidate difference strings for all four heavy pairs were rejected by both paths' admission gates. The check ran no symbolic evaluation or complete grading path; early string decisions and other inputs are outside it. The filters also reject ordinary expressions such as 2^{20}, so this is a compute-versus-admission trade-off, not evidence of superior accuracy.

Path	Time and execution boundary	Failure information
TRL	Calling thread; 10 s parse / 5 s compare on main thread, disabled otherwise	Unparseable reference: None; comparison timeout: 0
Ray example	Calling process; both library limits set to zero	Same reward conversion; no library deadline
verl main	Math-Verify is opt-in; prime_math has subprocess decoration plus the incompatible fallback above	Caught exceptions become false; no failure flag on those paths
OpenRLHF	Power filters; 120 s alarm on main thread, disabled otherwise	Filter refusal, timeout and other exceptions become false

Table 8: Pinned code comparison for the specified paths. The bounded fallback and admission-gate checks were executed; full framework configurations were not.

8 What a handling policy does to GRPO advantages

8.1 Illustrative groups

This subsection is an algebraic illustration, not a measured learning effect or a new theory of verifier noise. For a group of $G = 8$ binary rewards, fix the sample standard deviation and epsilon as

$$\bar{r} = G^{-1} \sum_i r_i, \qquad s^2 = (G-1)^{-1} \sum_i (r_i - \bar{r})^2, \qquad A_i = (r_i - \bar{r})/(s + 10^{-6}).$$

Assume the first answer would receive one if it were decided, but its observed undecided verdict is replaced by zero. Table 9 shows the advantage of that answer. Constant groups have $s = 0$ and all advantages zero under this convention.

Complete clean reward vector	Before zero replacement	After zero replacement
$(1, 1, 0, 0, 0, 0, 0, 0)$	1.620182	−0.353552
$(1, 0, 0, 0, 0, 0, 0, 0)$	2.474867	0
$(1, 1, 1, 1, 1, 1, 1, 1)$	0	−2.474867

Table 9: Conditional advantage changes for the first answer, at fixed standard-deviation and epsilon conventions (sample standard deviation, as `torch.std` in verl's GRPO estimator; values recomputed). No vector is inferred from the observed marginal failure rate.

A positive advantage can become negative or zero; a previously zero advantage can become negative. The group composition matters. In the first example, the remaining positive answer's advantage changes from 1.620182 to 2.474867. If instead the undecided row is excluded from *both* group moments and the loss, the remaining positive answer has advantage 2.267781, and the six negative answers each have −0.377963. This masking policy also changes the loss denominator: with equal response lengths and a token-mean loss over active answers, it is proportional to seven rather than eight responses. It is not equivalent to assigning the failed answer advantage zero after computing moments over all eight. The all-negative and all-positive remaining groups are constant and have zero advantages. These policies require an observable failure indicator; they do not supply the unknown clean reward, restore its learning signal, or establish a better trained model. Existing work already treats noisy rewards, degenerate groups and normalisation [3, 17, 37]. Plesner et al. [21] also distinguish advantage changes from clipped gradients and optimisation trajectories. The examples here illustrate conditional reward handling, not an optimizer update.

8.2 Handling policies on real labelled groups

The examples above use chosen vectors. To see how often such changes occur in realistic groups, we used the 72 groups of the labelled set (§5.7) that contain all eight responses to one prompt from the same 1.5B model; the other 2,969 groups are partial and cannot form GRPO statistics. Human labels serve as clean rewards; 26 groups are constant (9 all-wrong, 17 all-correct). We injected synthetic failures in three ways: independently per response (M1), only on label-1 responses (M2, failures that hide correct answers),

and for whole groups (M3, a trigger in the reference answer), at rates $p \in \{0.01, 0.05, 0.10, 0.25\}$, with 2,000 draws each and a fixed seed. Three policies were compared: P0 *zero-fill* (a failed row scores 0 and stays in statistics and loss, the behaviour of the tested `rate_limited` path); P1 *loss-only mask* (the zero stays in the group statistics, the row leaves the loss, as with a masked row under NeMo RL's default `include` policy [18]); and P2 *exclude* (the row leaves statistics and loss, as under NeMo RL's `exclude` policy and in iCoder-27B [36]). Advantages use verl's estimator; our implementation matched `compute_grpo_outcome_advantage` at `0c4c3d4` on 288 groups and sub-groups (maximum difference 2.9×10^{-7}), including its convention of mean 0 and std 1 for a single score.

Table 10: Offline handling-policy comparison on 72 labelled groups (576 responses); means per draw over 2,000 synthetic failure draws (Monte Carlo standard errors at most 0.7). *Own disagree*: responses in the policy's own loss whose advantage sign (negative, zero or positive) differs from the clean advantage; the evaluated rows differ between policies. The next three columns use one common set for all policies, the responses that did not fail (as many rows as P1/P2's loss): *0→nz* and *nz→0* count clean advantages that change from zero to non-zero and from non-zero to zero, and $|\Delta A|$ is the mean absolute advantage error. No surviving response changes sign between positive and negative under any policy. *Neg. correct*: label-1 responses with a negative advantage in the loss. *No signal*: of the 46 clean non-constant groups, those with no non-zero advantage left in the loss. Not a training result.

				Own	Common set (not failed)			Neg.	No
Failure	p	Policy	Loss rows	disagree	0→nz	nz→0	$\lvert\Delta A\rvert$	correct	signal
M1 independent	0.05	P0	576.0	57.9	39.1	1.2	0.060	17.2	0.19
		P1	547.2	40.4	39.1	1.2	0.060	0	0.19
		P2	547.2	5.0	0	5.0	0.024	0	0.75
	0.25	P0	576.0	183.6	88.0	6.3	0.275	85.5	1.26
		P1	432.2	94.3	88.0	6.3	0.275	0	1.26
		P2	432.2	22.5	0	22.5	0.098	0	4.44
M2 correct only	0.05	P0	576.0	57.7	38.9	1.4	0.060	17.2	0.21
		P1	558.6	40.3	38.9	1.4	0.060	0	0.21
		P2	558.6	1.4	0	1.4	0.013	0	0.21
	0.25	P0	576.0	184.2	88.3	8.5	0.264	85.6	1.29
		P1	488.6	96.8	88.3	8.5	0.264	0	1.29
		P2	488.6	8.6	0	8.5	0.076	0	1.29
M3 whole group	0.05	P0	576.0	18.2	0	0	0	0	2.28
		P1	547.2	0	0	0	0	0	2.28
		P2	547.2	0	0	0	0	0	2.28
	0.25	P0	576.0	92.0	0	0	0	0	11.49
		P1	432.1	0	0	0	0	0	11.49
		P2	432.1	0	0	0	0	0	11.49

Table 10 separates three effects. First, under zero-filling a failed correct response receives a negative advantage exactly when another correct response in its group did not fail; if every correct response in the group failed, all observed rewards are zero and so are the advantages. An exhaustive enumeration over all label and failure patterns of an eight-response group confirms this condition, and in the per-response settings (M1, M2) it holds for 98–99% of failed correct responses. Under whole-group failure (M3) no correct response receives a negative advantage; the group simply loses its signal. Second, on the responses that survive, which form the same set for every policy, zero-filling and loss-only masking are identical: the zero stays in the baseline. Neither flips a positive advantage to a negative one or vice versa; their disagreements are clean zeros that become non-zero (91–97% of P1's disagreements on the same draws; with binary rewards such zeros exist only in groups that were all-correct or all-wrong before failure) and, less often, non-zero advantages that become zero. Third, excluding failed rows from the statistics creates essentially no new signal (at most 0.007 such changes per draw) and has the smallest mean advantage error on the common set, but it zeroes more surviving advantages when failures are independent (5.0 versus 1.2 per draw at $p = 0.05$), leaves more groups with no signal (4.44 versus 1.26 at $p = 0.25$) and shrinks the loss denominator; with exactly one surviving response verl's convention assigns it its raw score as advantage, which occurred in at most 0.03 groups per draw. Smaller error on retained rows is therefore not by itself a better policy: the policies keep different rows and differ in how much signal they discard. For whole-group failures the policies agree on rewards and differ only in whether the failed group still counts in the loss denominator. None of these quantities is a gradient or a training outcome: clipping, token weighting and optimiser state are absent, the 72 groups are a stratified selection, and the

failures are injected. Which policy helps learning remains an empirical question.

9 Implications for reward-pipeline design

Verify the contract at the consumer boundary. The wrapper, manager and aggregator expose different views of the same attempt. A manager-side flag is useful only if the intended consumer can recover its row identity and meaning. The observed position-dependent outcomes justify testing sparse `error` and `timeout` keys at every row position, across mixed-success chunks, worker splits, padding/trimming boundaries and successive validation batches. The consumer still needs to define whether `None` means absent, unknown or a value excluded from statistics; after union-and-fill assembly an explicit `None` and an absent key are indistinguishable.

Treat a timeout change as a change in execution policy. The main replay contains rare limit-sensitive verdicts, but the shared-pool experiments show a separate consequence of extending a deadline. When an outer wait expires first, returning a score need not stop the child work; killing a shared pool can affect concurrent requests. Default-limit controls, serial calls and the longer-outer-wait controls distinguish these effects in the tested configuration. A deadline adjustment should therefore be checked for both verdict changes and effects on neighbouring requests, with resource interventions recorded separately.

Separate observability from a claim about learning. Retaining a failure flag establishes what happened to a scoring attempt. It does not establish the mathematical answer, determine the correct exclusion policy or demonstrate improved training: in our patched runs the flagged rows still receive reward 0 and still enter group statistics. Whether excluding them helps is not obvious in advance: if failures concentrate on longer, harder or particular correct responses, exclusion can shift the training distribution, and for an all-failed group the difference lies in loss denominators or resampling rather than in advantages. Table 10 quantifies the advantage-level side of this choice on real groups. The labelled replay also shows that answer representation can alter disagreement much more than increasing time limits in that selected set. A subsequent training evaluation would need to fix extraction, adjudicate disputed answers, and compare explicitly defined inclusion and normalisation policies. The current evidence provides concrete cases and regression requirements for that evaluation.

10 Related work

Verifier errors in RLVR. Cai et al. [3] model an imperfect verifier as a noisy reward channel with false-negative and false-positive rates and give population-level results and corrections. A fixed broken scoring worker under independent, uniform random assignment is a special case of such a channel. Their version 5 explicitly studies the bridge to group centering and implements corrections in a GRPO-style pipeline; it also distinguishes those guarantees from current-group standardisation and clipping (Appendices A–B). Consequently, a channel-level correction cannot simply be assumed to retain its guarantee under the pinned default estimator used here. Egashira et al. [6] inject systematic verification errors and observe delay, plateau or collapse; Plesner et al. [21] report modest mean performance losses in their multi-seed code experiments with resampled reward noise, while their uncertainty intervals allow larger losses; Zhang [38] contrasts training on code test suites with and without natural false positives. These works model or measure the effects of erroneous rewards. Our measurements locate the operations and aggregation policies that turn an unavailable decision into a reward or a batch exception.

Auditing verifiers and harnesses. Xin [35] audits four verifiers with metamorphic tests and separates returned verdicts from timeouts, errors and crashes; a follow-up [34] finds that verifier errors are correlated within GRPO groups and releases the human-labelled rollouts we reuse. Its discussion separates pooled dependence from prompt difficulty; our finite-corpus co-occurrence description retains that distinction. Verifier fuzzing [22], allocation and routing of repeated or multiple verifier calls [9, 10], reliability-first advantages [27] and harness checks [20] address neighbouring questions. Rollout Cards [16] includes a source audit of failure recording in 50 repositories and documents exception-to-zero coercion at aggregation boundaries. This motivates checking the consumer boundary as well as the scorer. Our

executed trace isolates a row-position effect in a particular manager and both of its aggregation paths; the 21 fixtures turn that trace into a concrete regression requirement.

The closest comparisons and the added evidence. The closest code-level records are #6545, #7151 and #6830/#6845 [1, 7, 8, 14]. #7151 also considers multiple workers and the fully asynchronous return path, and is the repair we port; #6830/#6845 concern `None` handling in validation metrics. Handling policies for ungraded responses are already documented elsewhere: Open Instruct defaults to a fallback zero, warns about its effect on relative advantages and offers a strict stop policy [2]; NeMo RL can exclude masked rows from GRPO group statistics [18]. Our study does not claim priority for the first-row-key mechanism, union-and-fill repair, cross-worker alignment, downstream `None` handling, or the observation that a fallback zero can bias group-relative advantages. It adds a trace from the specific #8015 worker failure through the manager's sparse flag to returned rewards and the validation consumer, re-executions of that trace on a second platform, fixed-response measurements under a stated unresolved rule, and matched timeout/recovery controls. The local prototype serves as a regression specification, not as a more complete framework fix.

Xin's verifier audit [35] uses certified answer transformations to distinguish returned verdicts from execution failures. We reuse the distinction while following runtime events through a wrapper, manager and consumer. Rollout Cards [16] already establishes that failure recording can disappear across layers; the row-position mechanism identifies the specific operation responsible in the tested configuration. Finally, the concurrent reset interaction was reported in the upstream discussion [25]. The natural heavy-input replay adds its manager-level consequence and matched default, serial, single-heavy and longer-wait controls. These comparisons delimit the added evidence without treating a known failure class as a new concept.

Failures in RL infrastructure. Zhang et al. [39] inject training faults and detect them from training curves. iCoder-27B [36] separates infrastructure from policy errors and excludes unjudgeable trajectories from both group statistics and loss normalisation (§4.4.3). This is a concrete prior handling policy, whereas our patches preserve metadata without implementing exclusion. On the optimisation side, Nie et al. [17] and Yang et al. [37] analyse degenerate groups and the bias of group-relative advantages; any statement about how undecided zeros shift degenerate-group rates must account for both. The pinned advantage examples here show conditional changes in finite reward vectors; they neither estimate the frequency of those vectors in training nor reproduce the theoretical results of these studies.

11 Limitations

Execution scope. The historical measurements use one Apple-silicon macOS machine; the new checks use one Linux container and cover the injection trace, the fixtures, the real-function suite, the labelled replay and a small set of heavy-input scenes, not the long-form corpus replays. Timing, signal delivery, process start methods and memory metrics differ between the two systems; the Linux port of the measurement guard reads resident plus swapped memory, which is not macOS physical footprint, and has no memory-pressure level. Generation and, except in the injection re-run, Ray transport were substituted; real scoring, manager and aggregation functions were exercised, but no complete trainer was run. The Math-Verify wrapper is optional in verl's default routing, and its adoption rate is unknown. The metadata prototype covers local batch assembly; downstream handling of missing values needs a separate check.

Reproducibility of the formal metric. The public package and the supplied raw rows let a reader verify file integrity, recount default-pass and pre-correction reference classes, reproduce $k = 1$ and the uncorrected $u = 9{,}573$, and re-execute the mechanism tests. They do not let a reader rebuild the corrected reference classes and hence $u = 18$: that needs the reference pass's parent, child and guard event logs. Recounting derived or compact rows is not an independent reproduction from raw events, and $u = 18$ remains a reported value.

Data and reference validity. The main corpus contains responses already filtered by a verifier or judge, with one answer sampled per problem. Its replay rates do not estimate training-time incidence. The longer-limit reference is the same verifier, and the separate human-labelled set is stratified, contains extracted answers, and was not independently re-adjudicated. Neither longer limits nor answer formatting

establishes mathematical truth, and a failure-event rate is not an error rate. Finite-sample recovery bounds and binomial working-model bounds answer different questions; the latter add homogeneity and independence assumptions not guaranteed by the corpus.

Supplementary analysis. The all-answer extension combines the audited main counts with later complement replays under a newer resource policy. Within-question co-occurrence uses exact problem text, not semantic deduplication, and groups have one to six answers rather than the eight-answer training groups used in the conditional calculations. Natural heavy-input scenes are selected diagnostics with explicitly changed limits. Their matched controls identify behaviour in those scenes rather than its frequency in unconstrained execution.

Learning effects. The advantage examples use declared reward vectors, variance conventions and a numerical stabiliser; the policy comparison uses 72 selected groups from one 1.5B model, human labels that were not re-adjudicated and synthetic failures, so its numbers describe advantage distortions in those groups, not natural rates. No policy gradients, training trajectories, or model-performance changes were measured. Observability is a software contract property; whether an exclusion or correction policy improves learning remains a separate empirical question.

12 Conclusion

At the tested verl head, the wrapper's failure-reporting policy composes poorly with the already reported first-row schema assumption: preserving an error at the manager boundary does not ensure that it survives batch assembly, and fixing assembly does not by itself ensure that it survives validation accumulation. Row position can determine whether the flag disappears or assembly fails. The 21 fixtures make reward preservation and metadata alignment explicit regression requirements; deployment still requires a defined missing-value and exclusion policy.

The fixed response corpora show rare limit-sensitive changes and leave some heavy inputs unresolved under a stated rule. Matched replay separates a returning timeout from unfinished background computation, and connects natural heavy inputs to an already reported reset interaction under relaxed diagnostic limits. These results support testing failures across the wrapper, manager, aggregation and consumer boundaries together. Once a failure is observable, the choice of handling policy matters at the advantage level: zero-filling penalises a hidden correct answer whenever a correct sibling survives, loss-only masking still shifts the survivors' baselines, and exclusion avoids that shift at the cost of retained rows and of signal in more groups. The results do not establish training incidence or a learning benefit from any of these policies.

A Reproduction

Evidence levels. (i) *Integrity:* `experiments/check_artifacts.py` verifies the 724 distributed files and 32 frozen report values; this is not replication. (ii) *Record recount:* the derived default rows, and the supplied pre-correction default and reference rows, were recounted with standard-library scripts; this reproduces Table 5, k and the uncorrected u, not the corrected classes. (iii) *Raw-event reconstruction:* not possible for the main reference pass with the supplied material. (iv) *Re-execution:* the injection trace under real Ray, the 21 fixtures, the real-function suite, the labelled replay and heavy-input scenes were executed on Linux. (v) *Offline:* the handling-policy comparison runs on the public labelled CSV without the pipeline. Commands, environments, outputs, a claim-to-evidence table and a script that checks the numbers in this manuscript against the result files are in the `audit/` directory of the source repository (https://github.com/feiiiiii5/rewardfail).

The companion archive, `rewardfail-companion.zip`, contains the source, compact reports, derived score records and selected event traces. Its `package_manifest.json` verifies the distributed files; the original-fingerprint index described below refers to the separately retained complete records. Public copies substitute portable path labels without changing reported scores or counts. The archive's README explains which checks run without the omitted datasets and full raw logs. The companion source pins the full dependency list in `requirements.lock`, including Math-Verify 0.9.0 and SymPy 1.14.0. The tested wrapper and manager are at verl pull-request head `0c4c3d4`; the comparison wrapper and prime_math audit are at `acad987`. OpenR1-Math-220k is at revision `e4e141ec9dea9f8326f4d347be56105859b2bd68`,

default configuration. The human-label CSV revision and provenance are cited in [33]. Its pinned repository carries an MIT licence, and the pinned OpenR1 data card declares Apache-2.0. These are upstream licence statements, not a separate audit of all source-data rights. The following records are kept separately:

- Injection mechanisms: `results/c1c/run1/` and `results/c1c/run2/`; original execution `cc87848`, with two recorded runs compared by `experiments/compare_runs.py`.
- Formal main measurement: `results/s2_main/report.json` and raw directory `main-20261007-202545`. Subsequent experiments never change this directory, the formal report or its thresholds.
- Supplementary diagnostics: `results/s2_u18/report.json`, raw directory `u18-20261008-104342`.
- Labelled-answer cross-check: `results/s2_e3/report.json`, raw directory `e3-20261008-codex`; driver at `2df9212`.
- All-answer extension: `results/s2_e2/report.json`, raw directory `e2-complement-20261008-codex`; driver at `4aa71b9`. The 100,034 complement answers use the original Parquet reader, separate answer-index identifiers and the original worker unchanged.
- Shared-pool controls and scenes: `results/s2_e1/selftest.json`, `results/s2_e1/report.json`, `report_eight.json` and `report_repeat.json` within that directory. Raw directories are `e1-core-20261008-codex`, `e1-eight-20261008-codex` and `e1-repeat-20261008-codex`; scene snapshots are `8b8321e`, `c026164` and `3bd7f94`, respectively. The committed reports are compact indexes with hashes of the complete raw records; they do not duplicate all fixed input strings.
- Sequential-parent and longer-outer controls: `results/s2_e1/report_controls.json`, raw directory `e1-controls-20261008-codex`; scene enumeration at `ca0503b`, execution identical to the earlier scene runner.
- Local batch-assembly proposal: `results/s2_e1/aggregation_contract.json`, generated by `experiments/s2_contract.py` at `ca0503b`. Pinned string-preprocessing checks: `results/s2_e1/normalization_filter.json`; these do not execute SymPy or a complete alternate verifier.

Each run records per-file SHA-256 code/data locks, specifications, child and parent events, pressure readings and intervention/restart history. Supplementary drivers use the locked worker's existing inline-text path; they do not edit the main worker or upstream wrapper files. Replay commands use fresh run directories and are launched with `nohup`, for example

```
nohup .venv/bin/python -I -B experiments/s2_e3.py \
  --run-dir <STAGE2>/runs/e3-new > <STAGE2>/runs/e3-new.log 2>&1 &
nohup .venv/bin/python -I -B experiments/s2_e1.py \
  --run-dir <STAGE2>/runs/e1-new > <STAGE2>/runs/e1-new.log 2>&1 &
nohup .venv/bin/python -I -B experiments/s2_e2.py \
  --run-dir <STAGE2>/runs/e2-new > <STAGE2>/runs/e2-new.log 2>&1 &
```

Run one measurement controller at a time. Green or warning memory pressure does not impose a static cap; critical pressure pauses/stops the run and is recorded as a local intervention, not as a natural verifier failure. The new drivers' own synthetic checks and raw-record reconstructions were recorded with their code/source locks. Reassembly of the new raw parent and child events and a separate count reconstruction matched the reported results; these checks use the same machine and underlying verifier.

The relative-path index `results/artifact_manifest.json` records SHA-256 hashes and byte sizes for 1,572 files, eight complete raw-run directories and 32 frozen report values. Its six configurable roots locate the companion repository, raw stage-two records, separate audits, data and the two upstream checkouts. The read-only `experiments/check_artifacts.py` verifies this inventory without scoring answers. Complete, missing-root and corrupt-index checks returned the declared pass, partial and fail statuses. File relocation was tested in temporary-directory fixtures; mapping the full inventory to different root aliases was tested on this host. Neither check is an independent experimental replication on another machine. Historical drivers and memory-pressure monitoring depend on the original macOS layout; a Linux port of the memory helpers, which reads a different memory metric and has no pressure level, is provided separately (`audit/companion_fixes/`).

The original main run's raw report is its historical execution report; `results/s2_main/report.json` is the audited reanalysis. Executed code snapshots and their locks take precedence over current source when interpreting a run. The inventory does not select redistribution licences or confer rights to upstream data or software. Source status was refreshed on 9 October 2026; the added sparse-metadata pull-request records were checked on 12 October 2026, and upstream pull-request status on 3 November 2026. Cited-paper metadata was initially checked on 8 October 2026 (and URL reachability by the author on 2 November 2026). A subsequent 3 November manuscript check accessed primary arXiv records and the full text of the closest overlapping studies; its coverage and version pins are recorded in `paper/LITERATURE_CHECK.md`. Original companion code is licensed under Apache-2.0; this does not replace upstream licences. This manuscript is licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0). Substantial AI assistance is disclosed in the author contribution statement.

Conflicts of Interest. The author declares no conflicts of interest.

References

[1] abinggo. verl pull request #6845: fix `process_validation_metrics` crash on None-filled sparse reward keys. https://github.com/verl-project/verl/pull/6845, 2026. Opened 2026-06-25; open when checked through the GitHub search API on 2026-11-03.

[2] Allen Institute for AI. Reward-service failure policies, in Open Instruct MILES documentation (data and evaluation). https://github.com/allenai/open-instruct/blob/65ff854953144ce5db5d152e17e79a794ac154f7/docs/miles/data-and-evaluation.md, 2026. Documentation source read at commit 65ff854 on 2026-11-03; the rendered site was not reachable from the checking environment. Title given descriptively.

[3] Xin-Qiang Cai, Wei Wang, Feng Liu, Tongliang Liu, Gang Niu, and Masashi Sugiyama. Reinforcement learning with verifiable yet noisy rewards under imperfect verifiers. arXiv:2510.00915, 2025. URL https://arxiv.org/abs/2510.00915v5. Version 5.

[4] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

[5] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948, 2025. URL https://arxiv.org/abs/2501.12948v2.

[6] Kazuki Egashira, Mark Vero, Jasper Dekoninck, Florian E. Dorner, Robin Staab, and Martin Vechev. Delay, plateau, or collapse: Evaluating the impact of systematic verification error on RLVR. arXiv:2605.02909, 2026. URL https://arxiv.org/abs/2605.02909v2. Version 2; checked 2026-10-08.

[7] fuergaosi233. Union-key assembly of reward extra information in verl pull request #6545. https://github.com/verl-project/verl/pull/6545, 2026. Closed without merging on 2026-06-02; uniform-key contract discussed by wuxibin89. Public record read 2026-10-12; status re-checked 2026-11-03.

[8] giladfrid009. verl issue #6830: `process_validation_metrics()` crashes on None-filled sparse `reward_extra_info` keys. https://github.com/verl-project/verl/issues/6830, 2026. Opened 2026-06-24; open when checked through the GitHub search API on 2026-11-03.

[9] Miaobo Hu, Shuhao Hu, Xiaobo Guo, Xin Wang, et al. ROUTEAUDIT: Interaction-aware identification for budgeted multi-verifier routing. arXiv:2610.02808, 2026. URL https://arxiv.org/abs/2610.02808v2.

[10] Miaobo Hu, Shuhao Hu, Xiaobo Guo, Xin Wang, et al. VStress: Correlation-aware auditing and adaptive budget allocation for repeated verifiers. arXiv:2609.36958, 2026. URL https://arxiv.org/abs/2609.36958v2.

[11] Hugging Face. Math-Verify. https://github.com/huggingface/Math-Verify, n.d.. Version 0.9.0, identical to commit ba3d3aaff23b3f4cac7a14672b4f6e293d97c98b.

[12] Hugging Face. OpenR1-Math-220k. https://huggingface.co/datasets/open-r1/OpenR1-Math-220k, n.d.. Configuration default, revision e4e141ec9dea9f8326f4d347be56105859b2bd68; Apache-2.0.

[13] Hugging Face. TRL: Transformer reinforcement learning. https://github.com/huggingface/trl, n.d.. Commit bbf50f4db3f212df0a1635a56a407d05579d385e.

[14] keepkeen. Sparse reward extra information across batch paths in verl pull request #7151. https://github.com/verl-project/verl/pull/7151, 2026. Reward-loop, agent-loop and fully asynchronous paths; head 96eead6f6a8d87f8c872fda263d3e3d41bf7cdc8; open and unmerged when read on 2026-10-12 and re-checked on 2026-11-03. Title given descriptively.

[15] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, et al. Tulu 3: Pushing frontiers in open language model post-training. arXiv:2411.15124, 2024. URL https://arxiv.org/abs/2411.15124v5.

[16] Charlie Masters, Ziyuan Liu, and Stefano V. Albrecht. Rollout cards: A reproducibility standard for agent research. arXiv:2605.12131, 2026. URL https://arxiv.org/abs/2605.12131v1.

[17] Wenhua Nie, Jianan Wu, Junlin Liu, Ziwei Li, et al. Gradient starvation in binary-reward GRPO: Why group-mean centering fails and why the simplest fix works. arXiv:2605.07689, 2026. URL https://arxiv.org/abs/2605.07689v1.

[18] NVIDIA NeMo RL contributors. Masked rewards in advantage statistics, in NeMo RL GRPO guide. https://github.com/NVIDIA-NeMo/RL/blob/db03c2c9c48c4b5adf9aa342ed8d31d03b09f29d/docs/guides/grpo.md, 2026. Documentation source read at commit db03c2c on 2026-11-03; the rendered site was not reachable from the checking environment. Title given descriptively.

[19] OpenRLHF contributors. OpenRLHF. https://github.com/OpenRLHF/OpenRLHF, n.d. Commit dc2a7ad326f619fc5a47737ea47d1920ce2c0b53, read 2026-10-08.

[20] Taiheng Pan. Probe the harness: Setup checks for stale-data RL comparisons in language models. arXiv:2610.02911, 2026. URL https://arxiv.org/abs/2610.02911v1.

[21] Andreas Plesner, Francisco Guzmán, and Anish Athalye. An imperfect verifier is good enough: Learning with noisy rewards. arXiv:2604.07666, 2026. URL https://arxiv.org/abs/2604.07666v2. Version 2; checked 2026-10-08.

[22] Jaideep Ray. Before the model learns the bug: Fuzzing RLVR verifiers. arXiv:2606.01066, 2026. URL https://arxiv.org/abs/2606.01066v1.

[23] Ray contributors. Reinforcement learning with TRL on Ray Train (example). https://github.com/ray-project/ray/blob/0fe2388b9d4e4cf541609b4a11d5d95e348f1288/doc/source/train/examples/transformers/transformer_reinforcement_learning/README.md, n.d. Commit 0fe2388b9d4e4cf541609b4a11d5d95e348f1288, read 2026-10-08; title given descriptively.

[24] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024. URL https://arxiv.org/abs/2402.03300v3.

[25] shaurya416 and ChdDongyang. Discussion of concurrent reset behaviour in verl pull request #8015. https://github.com/verl-project/verl/pull/8015#issuecomment-5829231207, 2026. Report on 25 September; author response on 28 September at https://github.com/verl-project/verl/pull/8015#issuecomment-5862189678. Read 2026-10-08.

[26] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, et al. HybridFlow: A flexible and efficient RLHF framework. arXiv:2409.19256, 2024. URL https://arxiv.org/abs/2409.19256v2.

[27] Wan Tian, Zhongyi Li, Xiang Xu, Minhao Zou, et al. STAR-GRPO: Canonical anchoring and reliability-first advantages against representation-dependent reward hacking. arXiv:2609.36900, 2026. URL https://arxiv.org/abs/2609.36900v1.

[28] verl contributors. verl pull request #5635. https://github.com/verl-project/verl/pull/5635, 2026.

[29] verl contributors. verl pull request #5839. https://github.com/verl-project/verl/pull/5839, 2026.

[30] verl contributors. verl issue #8011. https://github.com/verl-project/verl/issues/8011, 2026.

[31] verl contributors. verl pull request #8015. https://github.com/verl-project/verl/pull/8015, 2026. Head 0c4c3d40d46d5f47d1918c905641eea969b7fa09; open on 2026-10-09 and, with the same head, on 2026-11-03.

[32] verl contributors. verl reward routing and prime_math grader. https://github.com/verl-project/verl/blob/acad9875a8bdfc81afbcb0a50d146b2630f44093/verl/utils/reward_score/prime_math/grader.py, 2026. Routing, grader and py_functional.py read at the same pinned revision on 2026-10-08.

[33] Esther Xin. Human-labelled extracted answers for rlvr_group_correlation. https://github.com/ethxin0011/rlvr_group_correlation/blob/ccb330209370ddbbd76717658074abd54a7c45d0/README.md, 2026. CSV SHA-256 90fca106cb07f031a7ac169e009e6ab9cc0839126912218d0763b0ab6e037005; sampling and schema read 2026-10-08.

[34] Esther Xin. Are verifier errors independent within a GRPO group? evidence from Qwen2.5 rollouts. arXiv:2609.06386, 2026. URL https://arxiv.org/abs/2609.06386v1.

[35] Esther Xin. Where the verifier fails: A category-level audit of reward signals in RLVR. arXiv:2609.01354, 2026. URL https://arxiv.org/abs/2609.01354v1.

[36] Cheng Yang, Jiayang Lyu, Shangyuan Liu, Guibin Zhang, et al. iCoder-27B: Recursive AI-led development of frontier industrial coding model. arXiv:2609.29626, 2026. URL https://arxiv.org/abs/2609.29626v1.

[37] Fengkai Yang, Zherui Chen, Xiaohan Wang, Xiaodong Lu, et al. Your group-relative advantage is biased. arXiv:2601.08521, 2026. URL https://arxiv.org/abs/2601.08521v3.

[38] Chuyifei Zhang. When the reward suite is leaky: A preregistered causal contrast of natural verifier false positives in RLVR. arXiv:2607.11022, 2026. URL https://arxiv.org/abs/2607.11022v1.

[39] Lingzhe Zhang, Tong Jia, Yunpeng Zhai, Liancheng Fang, et al. Towards robust LLM post-training: Automatic failure management for reinforcement fine-tuning. arXiv:2605.04431, 2026. URL https://arxiv.org/abs/2605.04431v1.